KVM-Sentinel-SOC — التحليل التنفيذي الشامل

مركز عمليات أمنية ذكي مدمج مع مراقب الأجهزة الافتراضية (Hypervisor) — تحليل قائم على الأدلة المصدرية

كل ادعاء في هذا التقرير موثّق بمسار الملف ورقم السطر path:line. نطاق هذه المراجعة مقتصر على القراءة الساكنة للشفرة المصدرية؛ وأي عنصر يتعذّر التحقق منه سكونياً مُدرج في قسم «المجالات غير المتحقق منها».

1. الملخص التنفيذي (غير تقني)

يمثّل مشروع KVM-Sentinel-SOC معملاً تعليمياً وأمنياً متكاملاً مبنياً على ثلاث طبقات: نظام مضيف (Host) يعمل بتقنية KVM/libvirt، وآلة افتراضية ضيفة أولى تعمل كـ«فخّ» (Honeypot) مُتعمَّد إضعافه، وآلة افتراضية ثانية تحتضن مركز عمليات أمنية مصغّراً (SOC) مدعوماً بالذكاء الاصطناعي. يشنّ المهاجم (سواء أكان طالباً في قاعة الدرس أم أداة المحاكاة المحلية) هجمات على خدمة SSH في آلة الفخّ؛ فتقوم حاوية promtail بشحن السجلات فورياً إلى مخزن Loki في الآلة الثانية؛ وهناك يعمل خط أنابيب مكوّن من أربعة وكلاء برمجيين بلغة Python (Parseur ثم Détecteur ثم Analyste ثم Rapporteur) يفحص السجلات كل ثانيتين ويُطلق تنبيهاً عند رصد نمط هجومي، ثم يولّد تقرير حادثة مفصّلاً ويرسل إشعار HTTP إلى خادم التحكّم والموافقة (approval_daemon.py) على المضيف. يعرض هذا الخادم الأدلة الجنائية للمُشغّل البشري ويطرح سؤال تأكيد وحيداً [y/N] قبل تنفيذ أي إجراء. وعند الموافقة، يُنفَّذ أمران اثنان فقط عبر أداة virsh: أخذ لقطة فورية (snapshot) لحفظ الأدلة الجنائية، متبوعاً بفصل واجهة الشبكة المعزولة (NIC) لعزل الفخّ تماماً. ويُسجَّل كل قرار في سجل تدقيق بصيغة JSON-lines. يتلخّص المبدأ الحاكم للتصميم (وفق docs/ARCHITECTURE.md:14–15) في العبارة: **AI advises, human decides, hypervisor executes** (الذكاء الاصطناعي يوصي، والإنسان يقرر، ومراقب الأجهزة الافتراضية ينفّذ).

مواطن القوة المعمارية البارزة:

- **عزل شبكي حقيقي**: جسر شبكة الفخّ مجرّد من التوجيه (forwarding) ومن خدمة DHCP، كما أن الآلة الافتراضية الأولى (vm-honeypot) تعمل بلا بوابة افتراضية (host/networks/backend-isolated.xml:6–18) (default route).
- **جمع الأدلة الجنائية قبل الاحتواء**: إنشاء لقطة snapshot بالخيار الذرّي --atomic قبل فصل واجهة الشبكة (host/controller/approval_daemon.py:349–355).
- **سجل تدقيق أمني غير قابل للمحو (Append-only Audit Log)**: يُدوَّن كل قرار بشري في ملف JSONL مع فرض المزامنة الفورية على القرص عبر (host/controller/approval_daemon.py:177–186) fsync.
- **توثيق مستقل لكل حادثة**: إصدار تقريرين بصيغتَي Markdown وJSON لكل حادثة داخل مجلد reports بملكية المستخدم ذي المعرّف (cloud-init-vm2/user-data:78) UID 10002.
- **غلاف تشغيلي متكامل**: لوحة تحكم عبر tmux تضم 9 نوافذ و13 فحصاً آلياً لصحة الخدمات (scripts/).
- **وضع الاشتباك الحي (Live-Fire) للقاعات الدراسية**: تمرير حركة المرور عبر قواعد DNAT على واجهة المضيف لإتاحة الهجوم المباشر للطلاب في بيئة مضبوطة (host/provisioning/enable-live-fire.sh:130–143).
- **معالجة محلية بالكامل**: يتم الاستدلال اللغوي عبر النموذج الخفيف qwen2.5:0.5b المشغَّل بواسطة Ollama على المعالج المركزي (CPU) داخل VM2، مما يضمن عدم تسرّب أي بيانات خارج المعمل.

2. نظرة عامة على المعمارية

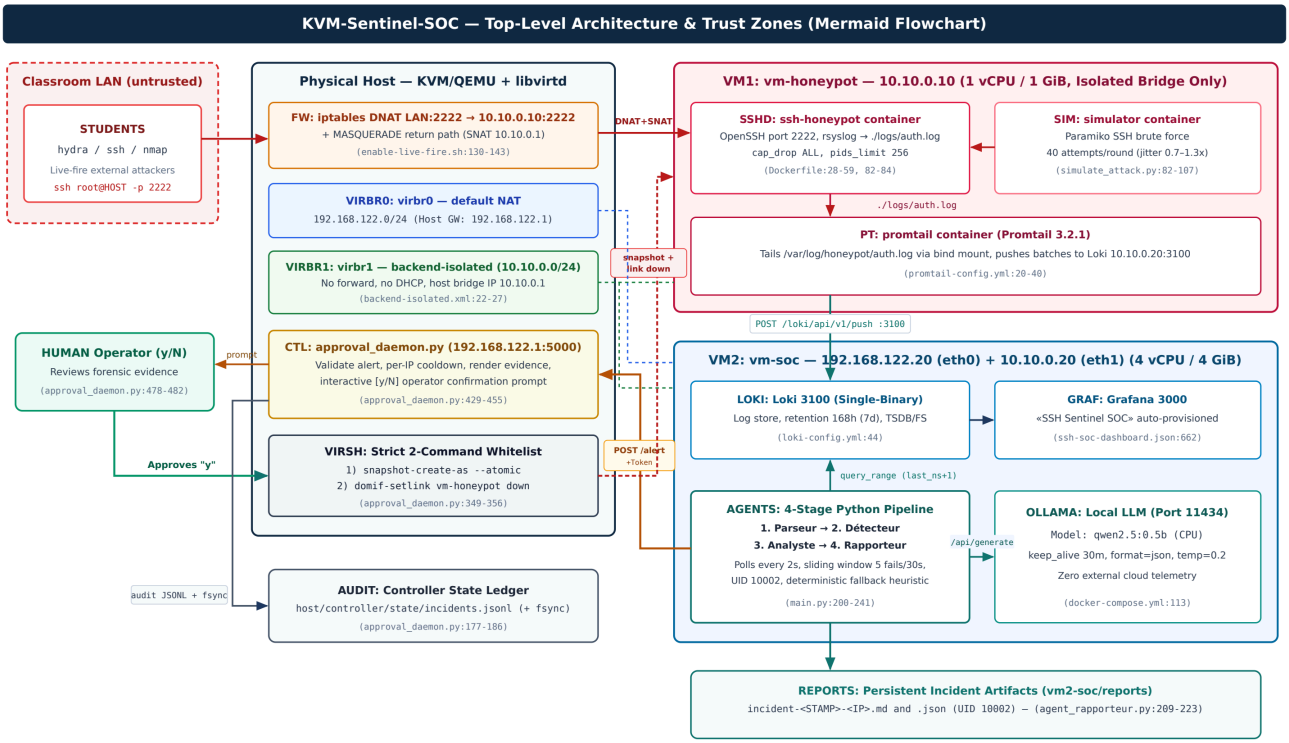
2.1 جدول المكوّنات الأساسية

المكوّن — الموقع في الشفرة المصدرية	الدور الوظيفي والمنفذ والتقنيات المستخدمة
`ssh-honeypot container` — `vm1-honeypot/docker-compose.yml:27–61` و `ssh-honeypot/Dockerfile:1–98`	حاوية فخّ SSH مُضعَّفة عمداً على المنفذ 2222 تكتب السجلات في `auth.log` عبر `rsyslog`؛ مع ضوابط تحصين صارمة: `cap_drop ALL` و`no-new-privileges` و`pids_limit 256`؛ مبنية على OpenSSH + `rsyslog`.
`promtail container` — `vm1-honeypot/docker-compose.yml:94–105` و `vm1-honeypot/promtail/promtail-config.yml:1–40`	شحن سجلات `auth.log` إلى خدمة Loki على العنوان `10.10.0.20:3100`؛ مع وسم السجلات بتصنيفات (`labels`) منخفضة التعدد (`Low Cardinality`)؛ يعمل بإصدار `Promtail 3.2.1`.
`simulator` — `vm1-honeypot/docker-compose.yml:63–92` و `vm1-honeypot/simulator/simulate_attack.py:1–218`	مولّد هجمات تخمين SSH حقيقية عبر مكتبة `paramiko` مضبوط بملف تعريف يدوي (`manual profile`)؛ يعمل على `python:3.12-slim` + `paramiko`.
`loki` — `vm2-soc/docker-compose.yml:25–45` و `vm2-soc/loki/loki-config.yml:1–80`	مخزن سجلات أحادي الملف التنفيذي (`single-binary`)، بفترة احتفاظ تمتد 7 أيام (`168h`)، وتخزين محلي `tsdb/fs`؛ يعمل بإصدار `Loki 3.2.1` على المنفذ 3100.
`grafana` — `vm2-soc/docker-compose.yml:47–78` و `vm2-soc/grafana/provisioning/`	لوحة المراقبة «SSH Sentinel SOC» المُجهَّزة تلقائياً بمعرّف مصدر بيانات ثابت `loki`؛ بيانات الدخول الافتراضية `admin/sentinel`؛ تعمل بإصدار `Grafana 11.2.2` على المنفذ 3000.
`ollama` و`ollama-init` — `vm2-soc/docker-compose.yml:80–119`	محرك استدلال محلي على CPU؛ مهلة إبقاء النموذج في الذاكرة `keep_alive 30m`؛ حاوية تهيئة أولية تسحب النموذج `qwen2.5:0.5b`؛ يعمل بإصدار `Ollama 0.5.4` على المنفذ 11434.
ملفات `agents` — `vm2-soc/agents/main.py:101–241` + 4 وكلاء	خط أنابيب يضم `Parseur` و`Détecteur` و`Analyste` و`Rapporteur` باستقصاء دوري (`Polling`) كل ثانيتين؛ مبني على `pydantic` + `Python 3.12` + `requests 2.9`؛ يعمل بمعرّف `UID 10002`.
`approval daemon` — `host/controller/approval_daemon.py:1–652`	مستقبِل Webhook على `192.168.122.1:5000`؛ يعرض الأدلة ويطلب موافقة المشغّل `[y/N]` قبل تنفيذ أمرَي `virsh` حصراً؛ يوثّق في سجل JSONL؛ مبني على `Flask 3.0.3` + `Rich`.
`restore-honeypot.sh` — `host/controller/restore-honeypot.sh:1–142`	استعادة الآلة الافتراضية إلى اللقطة المرجعية `clean-baseline` وإعادة تفعيل واجهة الشبكة المعزولة؛ مكتوب بـ `bash` + `virsh`.
`enable-live-fire.sh` — `host/provisioning/enable-live-fire.sh:1–226`	توجيه DNAT للمنفذ 2222 على المضيف نحو `10.10.0.10:2222` مع تفعيل MASQUERADE؛ يعتمد على `iptables` (قواعد موسومة بـ `kvm-sentinel-livefire`).

المكوّن — الموقع في الشفرة المصدرية	الدور الوظيفي والمنفذ والتقنيات المستخدمة
`shutdown-lab.sh` و `cleanup-lab.sh` — `host/provisioning/`	إيقاف آمن يحفظ الحالة التشغيلية مقابل أداة الهدم الكامل المُتعمَّد؛ مكتوب بـ `bash` + `virsh` + `docker`.
`backend-isolated` (`virbr1`) — `host/networks/backend-isolated.xml:1–27`	جسر شبكي معزول `10.10.0.0/24`: بلا توجيه (`forward`) وبلا DHCP؛ عنوان المضيف على الجسر `10.10.0.1`؛ معرّف بملف `libvirt network` XML.
`tmux-dashboard.sh` و `dashboard-status.sh` و `tmux-dashboard-stop.sh` — `scripts/`	منظومة التشغيل والمراقبة: 9 نوافذ، و13 فحص صحة، وإيقاف آمن يتحقق من تطابق `PID+STARTTIME`؛ تعتمد على `tmux` + `bash` + `curl`.

2.2 المخطط المعماري العام للنظام

يوضّح الشكل التالي البنية المعمارية الشاملة للمعمل وحدود الثقة بين الشبكة الصفّية الخارجية، والمضيف الفيزيائي، وآلة الفخّ (`vm-honeypot`)، وآلة مركز العمليات الأمنية (`vm-soc`):



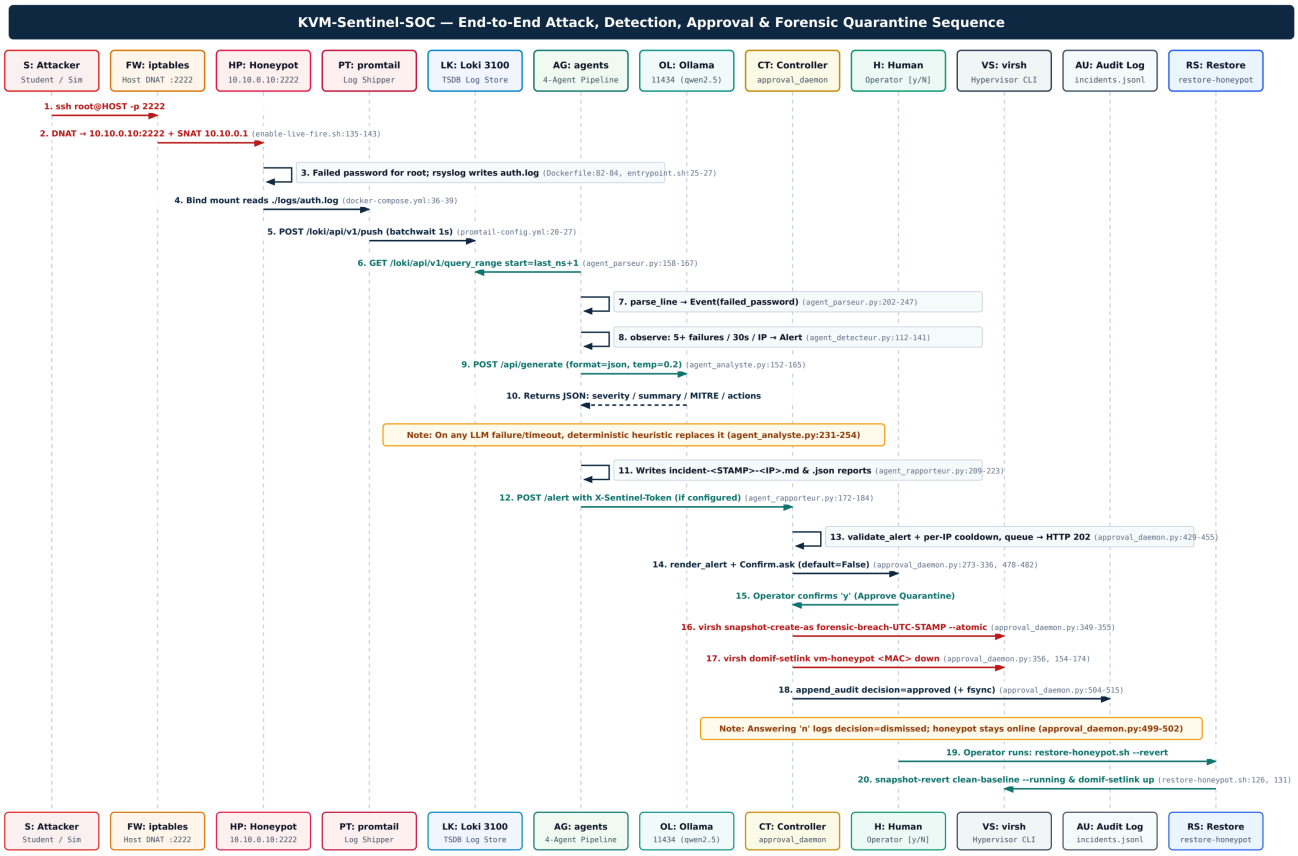
المخطط المعماري العام لمنظومة `KVM-Sentinel-SOC` وتدفقات الشبكة والتحكم (موثّق بمسارات الشفرة المصدرية).

تصميم الشبكات: تمثّل `virbr0` شبكة NAT الافتراضية الخاصة بـ `libvirt` على النطاق `192.168.122.0/24`، وعبرها تحصل واجهة `eth0` في الآلة `vm-soc` على مسار الخروج إلى الإنترنت (`cloud-init-vm2/network-config:11–24`). أما الشبكة `virbr1` فهي معرَّفة في الملف `backend-isolated.xml:22–27` دون أي عنصر `<forward>` — مما يعني أن `libvirt` لا يجري أي توجيه شبكي على الإطلاق (`CHANGELOG.md:38`، :6–12)، ويوثّق إزالة الخاصية `forward mode='none'` غير الصالحة في الإصدارات `libvirt 10+`). كما تخلو من خدمة DHCP (:13–15). يوفّر عنوان الجسر على المضيف `10.10.0.1` مسار العودة المباشر من المضيف إلى الآلتين الضيفتين (:16–18,26) مع نطاق محلي (:25 `sentinel.lab`). وتُثبَّت عناوين الآلات الافتراضية عبر `cloud-init`: حيث تأخذ واجهة `eth0` في `vm-honeypot` العنوان `10.10.0.10/24` بلا بوابة افتراضية (gateway) ولا مسارات توجيه (`cloud-init-vm1/network-config:9–21`)، بينما تأخذ واجهة `eth0` في `vm-soc` العنوان

192.168.122.20/24 مع البوابة 192.168.122.1 وخوادم DNS خارجية، وتأخذ واجهتها الثانية eth1 العنوان 10.10.0.20/24 بلا بوابة (cloud-init-vm2/network-config:10–33).

2.3 مخطط التسلسل الزمني الكامل للعمليات

يُبيّن مخطط التسلسل التالي دورة حياة الحادثة الأمنية خطوةً بخطوة، بدءاً من محاولة اختراق SSH، مروراً بشحن السجلات والفرز والكشف والتحليل اللغوي، وصولاً إلى قرار المشغّل البشري والعزل الجنائي ثم الاستعادة:



مخطط التسلسل الكامل (End-to-End Sequence Diagram) من لحظة الهجوم حتى العزل الجنائي والاستعادة.

3. تدفق البيانات من الطرف إلى الطرف (End-to-End Data Flow)

3.1 توليد الأحداث: من الهجوم إلى ملف auth.log

- **قناة المحاكاة المحلية**: زُوِّدت حاوية المحاكي simulator بملف تعريف تشغيلي يدوي (vm1-manual) (honeypot/docker-compose.yml:63–92) بحيث لا تقلع تلقائياً مع الحزمة، بل تُطلَق بأمر صريح مثل docker compose run --rm simulator، وهو الأمر ذاته الذي تطبعه نافذة shell في لوحة تحكم tmux.
- **آلية عمل المحاكي (simulator/simulate_attack.py)**: يولّد المحاكي موجات تخمين (Brute Force) حقيقية على بروتوكول SSH باستخدام مكتبة paramiko، مستعيناً بقائمتين تضمان 8 أسماء مستخدمين و10 كلمات مرور (:44–58)، مع حقن بيانات الاعتماد الصحيحة root:toor في كل محاولة تحمل الرقم 17 (:174–177)، وتطبيق معامل تذبذب عشوائي (jitter) وفق التوزيع المنتظم uniform(0.7, 1.3) على فترات الانتظار بين المحاولات (:192) لمحاكاة الإيقاع البشري، بواقع 40 محاولة لكل جولة، ويكون الهدف الافتراضي هو حاوية ssh-honeypot عبر الشبكة الداخلية (:115).
- **قناة الاشتباك الصفّي (Live-Fire)**: يزرع السكربت enable-live-fire.sh أربع قواعد في جدار الحماية iptables (:136–143) موسومة بالتعليق المرجعي (:20 kvm-sentinel-livefire)؛ حيث تُجري تحويلاً للوجهة (DNAT) على الحزم الواردة إلى المنفذ 2222 على واجهة المضيف لتتجه إلى 10.10.0.10:2222، مع تفعيل التنكّر (MASQUERADE / SNAT)

10.10.0.1) لضمان عودة الردود من آلة الفخّ المعزولة التي لا تملك بوابة افتراضية (المبرّر المعماري مشروح في 14–10:)، مع توفير الأمر الفرعي status للتحقق من حالة القواعد.

- **داخل حاوية الفخّ**: ضُبط خادم sshd على الإعدادات Port 2222 وPermitRootLogin yes وMaxAuthTries 3 وLogLevel VERBOSE (ssh-honeypot/Dockerfile:28–59)، مع لافتة ترحيب SSH تمويهية (77–62:)، وقاعدة في rsyslog تحوّل رسائل مرفق المصادقة إلى المسار /var/log/honeypot/auth.log (84–82:). ويتولى سكربت الإقلاع entrypoint.sh توليد مفاتيح المضيف (16:)، وفرض كلمة المرور الضعيفة root:toor (19:)، وتشغيل خدمة rsyslogd (26:)، وإطلاق عملية متابعة مستمرة tail -n0 -F على ملف auth.log (30:)، قبل استبدال العملية الحالية بـ exec sshd (33:). وتنشر حزمة compose المنفذ 2222:2222 (vm1-honeypot/docker-compose.yml:35) مع فرض قيود العزل cap_drop ALL وno-new-privileges+ (44:) وpids_limit 256).

3.2 من auth.log إلى مخزن Loki

- تراقب حاوية promtail (vm1-honeypot/docker-compose.yml:94–105) ملف auth.log عبر ربط المجلد المشترك ./logs (bind mount)، وتدفع السجلات إلى خدمة Loki على العنوان 10.10.0.20:3100 عبر الواجهة المعزولة eth1 للآلة vm-soc (promtail/promtail-config.yml:21)، بمعدل إرسال يبلغ ثانية واحدة (batchwait: 1s) أو عند بلوغ حجم الدفعة 1 ميغابايت (23–22:)، مع سياسة تراجع أُسّي (Exponential Backoff) تبدأ من 500ms وتصل إلى 5 دقائق بحدّ أقصى قدره 20 محاولة (27–24:).
- تُلحق كل دفعة بتصنيفات ثابتة منخفضة التعدد (job وhost و39–36:) (source)، مع تحديد مسار الالتقاط (/var/log/honeypot/auth.log :40).
- يُحفظ ملف مؤشر القراءة positions.yaml داخل المجلد المؤقت (/tmp :18–17) — وهو مسار غير مستديم؛ لذا فإن إعادة تشغيل حاوية promtail قد تؤدي إلى إعادة شحن سجلات قديمة، وتقع مسؤولية منع الازدواجية على عاتق وكلاء التحليل (انظر القسم 3.4).

3.3 مخزن Loki: مستقبِل محايد للسجلات

- يعمل Loki وفق الإعدادات في vm2-soc/loki/loki-config.yml:1–80 بنمط الملف التنفيذي الموحّد (single-binary) على المنفذ 3100، وبفترة احتفاظ تبلغ 168 ساعة (7 أيام)، مع تحديد سقف معدل الاستقبال عند 8 ميغابايت/ثانية وحدّ اندفاع (Burst) يبلغ 16 ميغابايت، وتعطيل المصادقة (auth_enabled: false، وهو إعداد مقبول ضمن الشبكة المعزولة)، في حين يظهر مكوّن القواعد ruler مفعّلاً في التكوين دون أي قواعد محمّلة — مما يجعله مكوّناً خاملًا حالياً (انظر القسم 9).
- تُمرَّر متغيرات البيئة إلى حاوية الوكلاء عبر vm2-soc/docker-compose.yml:134–150: فترة الاستقصاء POLL=2s، ونافذة الاسترجاع الأولي LOOKBACK=30s، ومهلة النموذج اللغوي OLLAMA_TIMEOUT=25s، وعتبة الكشف THRESHOLD=5، والنافذة الزمنية WINDOW=30s، وفترة التهدئة COOLDOWN=120s، وعنوان الإشعار WEBHOOK=http://192.168.122.1:5000/alert، بينما يُترك الرمز السري SENTINEL_TOKEN فارغاً بشكل افتراضي (149:) — وهو ما يمثّل مخاطرة أمنية موثّقة في القسم 6.
- يُضاف تعيين المضيف عبر extra_hosts: host-gateway (157–156:) لتوفير مسار اتصال محدد من حاوية الوكلاء نحو المضيف الفيزيائي.

3.4 من Loki إلى إطلاق التنبيه: وكيلا الفرز (Parseur) والكشف (Détecteur)

- يستعلم وكيل الفرز parseur عبر الدالة fetch (agent_parseur.py:158–198) من نقطة النهاية /loki/api/v1/query_range بدءاً من الطابع الزمني start=last_ns+1 وبحدّ أقصى 5000 سطر، ثم يطبّق التعبيرات النمطية (regexes) بترتيب صارم (66–49:) — بدءاً بنمط failed_password، ثم invalid_user، ثم accepted_password — بعد تمرير السجلات على مرشّح الضجيج NOISE_MARKERS (78–70:) لاستبعاد الرسائل غير الأمنية، ليُنتج كائنات أحداث مهيكلة وفق نموذج Event (91–81:).
- يُحفظ المؤشر الزمني last_ns ذرّياً على القرص عبر os.replace (155–146:)، ولكن لا يتم هذا الحفظ إلا بعد انتهاء معالجة الدفعة كاملةً في الحلقة الرئيسية (main.py:231–232) — مما يترك نافذة زمنية ضيقة قد تؤدي عند حدوث استثناء

إلى فقدان تنبيه أو تكراره (نُوقش في القسم 9).

- **منع الازدواجية (Deduplication)**: يحتفظ الوكيل بطابور مزدوج النهاية (deque) يختزن بصمات SHA-1 لآخر 20,000 سطر (194–191 :، 124:) لحماية خط الأنابيب من السجلات المعاد شحنها عند إعادة تشغيل promtail.
- **منطق المراقبة والكشف**: ينشئ وكيل الكشف detecteur نافذة زمنية منزلقة مستقلة لكل عنوان IP مصدر (agent_detecteur.py:95–99)، ويُخلي الأحداث المتقادمة عبر الدالة 110–102: (_prune). وفي الدالة observe 141–112:))، تُسجَّل عمليات الدخول الناجحة (accepted) في سجل تتبّع منفصل (120–118:)، بينما تقتصر مراكمة العتبة على أحداث 123–122: (failed_password)؛ وعند بلوغ 5 محاولات فاشلة خلال 30 ثانية من نفس العنوان، يُطلَق كائن تنبيه Alert، وتُفرَض فترة تهدئة (cooldown) مدتها 120 ثانية لهذا العنوان (135–133:) مع تفريغ نافذته الحالية (140:).
- **حساب درجة الخطورة (Severity Score)**: تُحسَب الدرجة داخل الدالة 164–151: (_build_alert) انطلاقاً من قيمة أساسية قدرها 5، ويُضاف إليها 1+ عند كل مضاعفة للعتبة (2× ثم 4× ثم 8×)، و1+ عند استهداف حساب عالي الأهمية من القائمة 38: (HIGH_VALUE_USERS)، و2+ عند رصد استخدام بيانات الاعتماد المكشوفة (root:toor).

3.5 من التنبيه إلى التقرير والتسليم: وكيلا التحليل (Analyste) والتوثيق (Rapporteur)

- يبني وكيل التحليل analyste مطالبة النموذج اللغوي بالاستناد إلى موجّه النظام SYSTEM_PROMPT (agent_analyste.py:38–47) وعقد المخرجات 60–49: (OUTPUT_CONTRACT)، ويستخلص كائن JSON بآلية متسامحة مع الضجيج النصي (98–72:) مع تعقيم السلاسل النصية (104–101:)، ويرسل طلب POST /api/generate إلى خدمة Ollama بالمعاملات: format=json وtemperature=0.2 وnum_ctx=2048 وnum_predict=700 (165–160:) بعد تسخين مسبق للنموذج عبر 128–114: (warm_up). وفي حال حدوث أي فشل في النموذج اللغوي أو انتهاء مهلته أو إرجاع JSON غير صالح، ينتقل الوكيل تلقائياً إلى دالة استدلالية حتمية (254–231:) (Deterministic Heuristic Fallback) تُسند تقنية MITRE ATT&CK رقم T1110.001 (تخمين كلمات المرور)، وتضيف T1078 (الحسابات الصالحة) عند رصد نجاح الدخول ببيانات root:toor.
- يتولى وكيل التوثيق rapporteur صياغة تقرير كامل بصيغة Markdown (agent_rapporteur.py:70–133) مستخدماً الدالة safe_token لتقنيع السلاسل الطويلة أو المشبوهة (52–50:)، ويولّد قائمة الأوامر المقترحة proposed_commands مقصورةً دائماً على أمرين اثنين (151–145: virsh snapshot-create-as ... forensic-breach-UTC-STAMP: وvirsh domif-setlink ... down، كما يبني حمولة JSON (163–153:) بطابع زمني قياسي UTC، ويكتب الملفين على القرص بطريقة ذرّية (169–165:). وتنفّذ الدالة deliver حتى 4 محاولات إرسال نحو المضيف بتراجع أُسّي 2^attempt مع الامتناع عن إعادة المحاولة عند تلقي أخطاء العميل 206–172: (HTTP 4xx)، وتُرفق الترويسة X-Sentinel-Token فقط إذا كان الرمز مضبوطاً (175–173:).
- تحرص الدالة 238–209: (publish) على كتابة ملفَّي .md وjson. في مجلد reports أولاً دون أن تتوقف عند فشل التسليم الشبكي — مما يضمن حفظ التقارير الجنائية محلياً على القرص تحت جميع الظروف.

3.6 من التقرير إلى القرار البشري والتنفيذ الجنائي

- يعمل خادم الموافقة host/controller/approval_daemon.py عبر إطار Flask (create_app في 461–388:)، ويُقيّد الاستماع الشبكي (bind) على العنوان الخاص بالمضيف 192.168.122.1 رافضاً العمل على العنوان المفتوح 0.0.0.0 (579–567:))، ويحدّد الحجم الأقصى للطلب بـ 64 KiB (MAX_CONTENT_LENGTH = 390:)، ويتحقق من رمز المصادقة بطابق زمني ثابت عبر 396–392: (hmac.compare_digest) مع الفحص داخل المسار POST /alert في 433–431:)، ويطبّق فترة تهدئة (cooldown) مدتها 120 ثانية لكل عنوان مصدر (450–442:)، ثم يودع التنبيه في طابور داخلي سعته 12 عنصراً مع إرجاع الاستجابة HTTP 202 Accepted فوراً (455–452:)، ليعالجه خيط خلفي (daemon thread في 607– 613) وحلقة استهلاك تفاعلية (637–629:).
- **التحقق الصارم من المدخلات (validate_alert في 267–213:)**: يفحص وجود جميع الحقول الإلزامية (REQUIRED_FIELDS :70–80)، ويتحقق من صحة عنوان IP عبر مكتبة ipaddress القياسية (244–240:)، ويربط اسم الآلة الافتراضية بقائمة النطاقات الموثوقة (247–246:)، ويحصر درجة الخطورة severity ضمن النطاق الصحيح (249– 254)، ويطابق معرّف التنبيه alert_id بتعبير نمطي صارم (266–263:).

- **العرض التفاعلي (`render_alert` في 273–336:)**: يركّب خادم الموافقة أمرَي `virsh` المسموح بهما داخلياً من معاملات موثوقة (318–323:)، بينما يعرض حقل `proposed_commands` الوارد في التقرير لأغراض القراءة والمراجعة فقط دون أن يمرّره للتنفيذ إطلاقاً (332–336:).
- **تفاعل المُشغّل البشري والتنفيذ (`perform_quarantine` في 339–382:)**: يطرح الخادم سؤال التأكيد عبر `Confirm.ask` بقيمة افتراضية رافضة (`default=False` :478–482) — ما لم يُفعَّل خيار الموافقة الآلية `auto_approve` صراحةً (472–474:). وعند موافقة المشغّل بإدخال y، ينفّذ الخادم أولاً أمر أخذ اللقطة الجنائية `virsh snapshot-create-as ... forensic-breach-UTC-STAMP --atomic` (349–355:)، ثم يستخرج عنوان MAC لواجهة الشبكة المعزولة عبر `domiflist` (154–174:)، ويتبعه بالأمر الثاني `virsh domif-setlink ... down` (356:)، مع دعم وضع المحاكاة الجافة `dry-run` (358–364:). وفي الختام، تستدعي `process_alert` (467–516:) الدالة `append_audit` (177–186:) لتسجيل القرار (`approved` أو `dismissed` في 504–515:) مع فرض `os.fsync`.
- **إجراءات الاستعادة (`host/controller/restore-honeypot.sh`)**: يستخرج السكربت عنوان MAC لواجهة الفخّ عبر `resolve_mac` (73–87:)، وينفّذ دالة الاستعادة `revert` (121–141:) التي تسترجع اللقطة المرجعية بالأمر `virsh snapshot-revert ... clean-baseline --running` (126:) وتعيد تفعيل واجهة الشبكة بالأمر `virsh domif-setlink ... up` (131:).

4. تفكيك المكوّنات: التزويد الآلي والطبقات المعمارية

4.1 طبقة النظام المضيف (`NixOS Host Layer`)

- يُفعّل ملف التكوين `host/nixos/virtualization.nix` خدمة `virtualisation.libvirtd` ويمنح الصلاحيات لأعضاء المجموعة المعتمدة (`host/nixos/virtualization.nix:75`)، بينما يربط ملف `flake.nix` حزمتَي `docker` و`libvirtd` ضمن بيئة النظام المضيف.

4.2 سكربت النشر والتزويد (`deploy-lab.sh`)

- **الفحوص الاستباقية (`preflight` في `host/provisioning/deploy-lab.sh:169–208`)**: يتوقف السكربت مبكراً عند غياب أي متطلب أساسي؛ إذ يتحقق من وجود الأدوات التنفيذية، وصلاحية الاتصال بـ `virsh`، وجاهزية الشبكات، وتوفّر الحد الأدنى من الذاكرة الحرة المقدّر بنحو (196:) `5500 MiB`).
- **تنزيل الصورة الأساسية**: يسحب السكربت صورة `Ubuntu 24.04 (Noble)` السحابية المثبّتة ويتحقق من سلامتها عبر بصمة SHA256 (245–267:).
- **طبقة التراكب (`QCOW2 Overlay`)**: ينشئ أقراص `qcow2` خفيفة تعتمد على الصورة الأساسية كملف مرجعي (`backing file`) باستخدام `qemu-img` (269–281:).
- **توليد ملفات التهيئة (`render_user_data` في 304–390:)**: يجهّز ملفات `user-data` و`meta-data` و`network-config` لكل آلة ضيفة. وفي `cloud-init-vm1/user-data`، يُضبط حساب الإدارة `soc:sentinel` (السطر 34)، ويُنسخ سكربت إشارة الجاهزية (`beacon`) إلى `/usr/local/bin` ليستمع على `10.10.0.10:9999` (`cloud-init-vm1/user-data:75–96`)، بينما تنفّذ قائمة `runcmd` (98–120:) حلقة تحقق تنتظر إقلاع حاوية `ssh-honeypot` بنجاح قبل كتابة ملف الجاهزية وتشغيل وحدة `systemd` المسماة `kvm-sentinel-ready`.
- **توليد صورة التهيئة (`Seed ISO`)**: تُحزَم ملفات `cloud-init` في صورة قرص بمعرّف الكتلة `cidata` عبر أداة `genisoimage` (392–417:).
- **تثبيت الآلة الأولى (`virt-install` `VM1` في 429–446:)**: تُنشأ بمواصفات `1 vCPU` وذاكرة `1024 MiB`، مع ربط قرص التراكب وشبكتَي `default` (مؤقتاً للتزويد) و`backend-isolated`، مع الخيار `--noautoconsole`.
- **تثبيت الآلة الثانية (`virt-install` `VM2` في 455–473:)**: تُنشأ بمواصفات `4 vCPU` وذاكرة `4096 MiB` (ملاحظة: يذكر ملخص الطباعة النهائي في السكربت الرقم `5120 MiB` لآلة VM2 عن طريق الخطأ — انظر القسم 9).

4.3 الانتظار والتحضير النهائي بعد التزويد

- تنتظر الدالة `wait_for_cloud_init` (:513–533) إشارة beacon عبر المنفذ `9999 TCP` — والتي لا تنطلق إلا بعد التحقق الفعلي من عمل حاوية `ssh-honeypot` (`runcmd`:98–120).
- **فصل واجهة التزويد المؤقتة (`detach_bootstrap_nic` في :535–552)**: تُزال واجهة NAT المؤقتة من VM1 فور اكتمال التزويد الأولي، فتبقى آلة الفخّ محصورةً كلياً داخل الشبكة المعزولة `backend-isolated`.
- **أخذ اللقطة المرجعية (`clean-baseline` في :554–565)**: تُؤخذ لقطة نظيفة للآلة VM1 لاستخدامها لاحقاً عند الاستعادة.
- **الملخص الختامي (:566–586)**: يطبع السكربت بطاقة الحالة النهائية (ويظهر فيها التفاوت المذكور حول رقم الذاكرة 5120 MiB — انظر القسم 9).

4.4 طبقة الفخّ (`vm-honeypot`) بالتفصيل

- **حزمة `vm1-honeypot/docker-compose.yml`**: تنشر حاوية الفخّ (:27–61) المنفذ 2222:2222 (:35) وتشارك مجلد السجلات `./logs` عبر (:36–39 `bind mount`)؛ وتفرض ضوابط التحصين `cap_drop ALL` و`no-new-privileges`+و`pids_limit 256` (:44)؛ وتربط الحاويات بشبكة داخلية `honeypot-net` على النطاق `172.31.240.0/24` (:116)؛ وتضم حاوية المحاكي تحت ملف التعريف اليدوي (`manual` :68–69) إلى جانب حاوية `promtail` (:94–105).
- **ملف البناء `ssh-honeypot/Dockerfile`**: يضبط خيارات sshd الضعيفة عمداً (`Port 2222` :28–59)، و`PermitRootLogin yes`، و`MaxAuthTries 3`، و`LogLevel VERBOSE`)، ويضع لافتة SSH تمويهية (:62–77)، ويضيف قاعدة rsyslog التي تفرّغ سجلات المصادقة في `auth.log` (:82–84).
- **سكربت البدء `ssh-honeypot/entrypoint.sh`**: يولّد مفاتيح المضيف عبر (`ssh-keygen -A` :16)، ويفرض كلمة المرور (`root:toor` :19)، ويشغّل خدمة (`rsyslogd` :26)، ويطلق `tail -n0 -F` على ملف (`auth.log` :30)، ثم ينفّذ `exec` (`sshd` :33).
- **أداة المحاكاة `simulator/simulate_attack.py`**: تختبر 8 أسماء مستخدمين و10 كلمات مرور (:44–58)، وتدرج المحاولة الناجحة `root:toor` في كل محاولة رقم 17 (:174–177)، مع تذبذب زمني (`uniform(0.7, 1.3)` :192)، وتستهدف افتراضياً العنوان (`ssh-honeypot` :115).

4.5 طبقة مركز العمليات الأمنية (`vm-soc`) بالتفصيل

- **حزمة `vm2-soc/docker-compose.yml`**: تشغّل خدمة loki بالإصدار `3.2.1` على المنفذ 3100 مع فحص جاهزية (`healthcheck`) على المسار `/ready` (:25–45)، وتفاصيل الفحص في (:38–44)؛ وخدمة grafana بالإصدار `11.2.2` وبيانات الدخول (`admin/sentinel` :47–78)؛ وخدمة ollama بالإصدار `0.5.4` مع الإعدادات `KEEP_ALIVE=30m` و`NUM_PARALLEL=1` (:89–92)؛ وحاوية التهيئة الأحادية `ollama-init` التي تسحب النموذج `qwen2.5:0.5b` مرة واحدة عند الإقلاع (:113)؛ وحاوية الوكلاء agents بمتغيرات بيئتها (:134–150) وربط (`extra_hosts` :156–157).
- **إعدادات `vm2-soc/loki/loki-config.yml:1–80`**: نمط أحادي (`single-binary`) على المنفذ 3100، واحتفاظ لمدة 168 ساعة، وتخزين `tsdb/fs`، وسقف استقبال 8 ميغابايت/ثانية بحدّ اندفاع 16 ميغابايت، وتفعيل `structured_metadata: true`، مع بقاء قسم `ruler` مفعّلاً دون قواعد (انظر القسم 9).
- **تزويد لوحات Grafana في `vm2-soc/grafana/provisioning/`**: يثبّت مصدر البيانات بمعرّف `uid: loki`، ويربط مزوّد اللوحات بملف اللوحة `ssh-soc-dashboard.json` الذي يضم 10 لوحات فرعية بتحديث تلقائي كل 10 ثوانٍ، معتمداً على استعلامات `count_over_time` لعبارة `Failed password` واستخراج العنوان المصدر `src_ip` عبر التعبيرات النمطية.

4.6 بيئة تشغيل الوكلاء (`vm2-soc/agents/main.py`)

- **هيكل الإعدادات (`Config` في :101–117)**: يقرأ متغيرات البيئة عبر دوال مساعدة (:48–61)، ويوفّر دالة تحويل الطوابع الزمنية (`iso_to_ns` :64–69)، ومؤشر النبض (`heartbeat` :72–79)، وطابور قياسات الأداء بسعة 50 عينة (:141)، ويضع قيمة افتراضية لمهلة `ollama_timeout` في الشفرة قدرها 180 ثانية (:109) مقابل 25 ثانية في ملف `docker-compose.yml` — وهو تفاوت موثّق في القسم 9.

- **حلقة التشغيل الرئيسية (:197–239)**: تبدأ بتسخين النموذج اللغوي عبر warm_up (:191–196)، وتطبّق سياسة تراجع أُسّي عند الأخطاء تبدأ من ثانية واحدة وتتضاعف (×2) حتى سقف 30 ثانية (:197–215)، وتستدعي الدالة handle_alert (:166–188)) لمعالجة كل تنبيه.
- **معالجة التنبيه (handle_alert)**: تمرّر التنبيه إلى وكيل التحليل analyste، ثم إلى وكيل التوثيق rapporteur لكتابة التقرير وإرساله إلى المضيف، مع طباعة تحذير مسبق عند ترك SENTINEL_TOKEN فارغاً (:162–163)، وتحديث ملف النبض heartbeat بعد كل دفعة (:233)، والانتظار وفق فترة الاستقصاء (:238).
- **تثبيت المؤشر (Cursor Commit)**: لا يُثبَّت المؤشر الزمني last_ns على القرص إلا بعد اكتمال معالجة جميع أحداث الدفعة (:231–232).

4.7 أدوات التحكّم والتشغيل على المضيف

- **خادم الموافقة host/controller/approval_daemon.py**: يضبط الإعدادات في Config (:105–124) بفترة تهدئة 120 ثانية ومسار التدقيق state/incidents.jsonl؛ وينفّذ أوامر virsh عبر مصفوفة معاملات argv مباشرة دون وسيط shell (:137–151)؛ ويستخرج عنوان MAC عبر resolve_nic_mac (:154–174)؛ ويكتب سجل التدقيق عبر append_audit مع فرض fsync (:177–186)؛ ويمنع الاستماع على 0.0.0.0 (create_app :388–461) مشغّلاً خادم Flask في خيط خلفي (:607–613) مع حلقة استهلاك الطابور (:629–637).
- **سكربت الاستعادة restore-honeypot.sh**: يستخرج عنوان MAC (:73–87)، ويعيد الآلة إلى اللقطة النظيفة clean-baseline عبر snapshot-revert --running (:126) ثم يفعّل واجهة الشبكة بالأمر domif-setlink up (:131).
- **سكربت الاشتباك الحي host/provisioning/enable-live-fire.sh**: يدير قواعد iptables الموسومة (:20، :136–143) مع أمر الفحص status.
- **الإيقاف والهدم**: يوقف shutdown-lab.sh الحزم والآلتين الافتراضيتين مع الحفاظ على حالتهما، بينما صُمّم cleanup-lab.sh ليكون أداة الهدم الكامل المُتعمَّد.
- **منظومة لوحة التحكم scripts/tmux-dashboard.sh**: تعرّف سجل المكوّنات (:223–260)، وتنشئ 9 نوافذ (:449–537)، وتحفظ ملفات PID مقترنة بوقت بدء التشغيل (:572–592)؛ بينما يجري dashboard-status.sh ثلاثة عشر فحصاً لصحة النظام (:180–454)؛ ويحمي tmux-dashboard-stop.sh من مخاطر إعادة تدوير معرّفات العمليات عبر مطابقة الزوج PID+STARTTIME (:129–133) مرسلاً إشارة SIGTERM قبل SIGKILL مع دعم الخيارين --stop-stacks و--stop-vms.

5. منطق الكشف والذكاء الاصطناعي

5.1 طبقة الفرز (Parseur)

- **ترتيب التعبيرات النمطية حسب الدقة (agent_parseur.py:49–66)**: يفحص الوكيل نمط failed_password أولاً، يليه invalid_user، ثم accepted_password؛ بحيث تُفحص الأنماط من الأكثر تخصيصاً إلى الأعمّ دون أن يحجب نمطٌ عامٌّ نمطاً أدقّ منه.
- **تصفية الضجيج (NOISE_MARKERS في :70–78)**: يستبعد الوكيل رسائل قطع الاتصال (disconnect)، وتبادل اللافتات (banner exchange)، ورسائل systemd، وفحوص الصحة الذاتية قبل دخولها إلى خط الكشف.
- **نموذج الحدث (Event في :81–91)**: يصنّف السجلات إلى أنواع محددة (failed_password :40–46) وinvalid_user وaccepted_password وغيرها.
- **منع التكرار وحفظ المؤشر**: يعتمد طابوراً لبصمات SHA-1 بسعة 20,000 عنصر (:124، :191–194)، ويستعلم من Loki عبر query_range بدءاً من last_ns+1 بسقف 5000 سطر (:158–198)، مع حفظ المؤشر ذرّياً عبر os.replace (:146–155).

5.2 طبقة الكشف (`Détecteur`)

- **النافذة الزمنية المنزلقة**: يحتفظ الوكيل بنافذة منزلقة مدتها 30 ثانية لكل عنوان IP (`agent_detecteur.py:95–99`)، وينظّفها باستمرار عبر الدالة `_prune` (`:102–110`).
- **منطق الرصد (`observe` في `:112–141`)**: تُسجَّل عمليات الدخول الناجحة (`accepted`) في هيكل مستقل (`:118–120`) لربطها بالسياق الجنائي، بينما تقتصر تعبئة نافذة العتبة على أحداث `failed_password` (`:122–123`)؛ وعند تجمّع 5 إخفاقات خلال 30 ثانية يُطلَق تنبيه `Alert`، مع فرض فترة تهدئة مدتها 120 ثانية (`:133–135`) وتفريغ النافذة (`:140`).
- **تتبّع الحسابات الحساسة**: تضم القائمة `HIGH_VALUE_USERS` (`:38`) الحسابات ذات الأهمية العالية التي يرفع استهدافها درجة الخطورة؛ كما أن رصد نجاح الدخول ببيانات `root:toor` يُعامَل بوصفه اختراقاً مؤكداً للفخّ.
- **معادلة حساب الخطورة (`_build_alert` في `:151–164`)**: تبدأ من الأساس 5، وتضيف 1+ عند كل مضاعفة للعتبة (/2×، 4×/8×)، و1+ لاستهداف حساب عالي القيمة، و2+ عند استخدام بيانات الاعتماد المكشوفة (`root:toor`) — وتُقيَّد النتيجة النهائية دائماً ضمن النطاق المعتمد `[1, 10]` مع إرفاق المبررات النصية المقروءة.

5.3 طبقة التحليل الذكي (`Analyste`)

- **هندسة المطالبة وعقد المخرجات**: يجمع الوكيل بين موجّه النظام (`SYSTEM_PROMPT` في `agent_analyste.py:38–47`) وعقد المخرجات (`OUTPUT_CONTRACT` في `:49–60`)، ويُلزم المخرجات بالحقول الآتية: درجة الخطورة `severity` كعدد صحيح في النطاق 1–10، وملخص نصي وجيز `summary`، وقائمة تقنيات `mitre_attack`، وقائمة الإجراءات المقترحة `actions` المحصورة ضمن القائمة البيضاء (`ALLOWED_ACTIONS` في `:35`).
- **استخلاص وتعقيم المخرجات**: يستخرج الوكيل كائن JSON بمرونة (`:72–98`) ويعقّم النصوص من المحارف غير الآمنة (`:101–104`)، بينما تبني الدالة `build_prompt` (`:131–149`) سياق الحادثة شاملاً النافذة الزمنية وعينات السجلات.
- **استدعاء النموذج اللغوي (`:152–195`)**: يرسل طلب `POST /api/generate` بالمعاملات `format=json` و`temperature=0.2` و`num_ctx=2048` و`num_predict=700` (`:160–165`)، بمهلة زمنية تُقرأ من البيئة (`25s` في `docker-compose.yml`، و`180s` كقيمة افتراضية في `main.py:109`)، في حين تقوم الدالة `warm_up` (`agent_analyste.py:114–128`) بتسخين النموذج مسبقاً عند بدء التشغيل.
- **آلية الاستدلال الاحتياطية الحتمية (`Heuristic Fallback` في `:231–254`)**: تعمل كشبكة أمان حتمية بالكامل — إذ تُسند المعرّف `T1110.001` لكل هجوم تخمين، وتضيف `T1078` عند رصد الدخول بـ `root:toor`، مع تطبيع الدرجة وحصرها عبر `_normalize` (`:202–203`).
- **الأهمية المعمارية**: يضمن هذا التصميم عدم توقف خط الكشف والاستجابة أبداً بسبب بطء المعالج المركزي أو تعثّر النموذج اللغوي الصغير؛ فالذكاء الاصطناعي يُثري التقرير السياقي، لكن المسار الحتمي يضمن استمرار العمل في جميع الحالات.

5.4 طبقة التقرير والإرسال (`Rapporteur`)

- **بناء التقرير (`agent_rapporteur.py:70–133`)**: يولّد تقريراً بصيغة Markdown بأقسام قياسية ثابتة تشمل: ملخص الحادثة، والأدلة الجنائية، والخط الزمني، وتصنيف MITRE ATT&CK، والأوامر المقترحة.
- **تقنيع السلاسل النصية (`safe_token` في `:50–52`)**: يقتطع ويقنّع السلاسل الطويلة أو المشبوهة لمنع تشويه تنسيق التقرير أو حقن محتوى مضلّل.
- **الأوامر المقترحة (`proposed_commands` في `:145–151`)**: تُقيَّد دائماً بالثنائية الثابتة: `virsh snapshot-create-as ... forensic-breach-UTC-STAMP` و`virsh domif-setlink ... down`.
- **الكتابة الذرّية والتسليم الشبكي**: يبني الوكيل حمولة JSON (`:153–163`)، ويكتب الملفات ذرّياً (`:165–169`)، ويرسل الإشعار عبر `deliver` بـ 4 محاولات مع تراجع أُسّي `2^attempt` دون إعادة المحاولة على أخطاء 4xx (`:172–206`)، مع إرفاق ترويسة `X-Sentinel-Token` عند ضبطها (`:173–175`)، بينما تضمن الدالة `publish` حفظ ملفَّي `.md` و`.json` على القرص دائماً (`:209–238`).

6. نموذج الأمان وحدود الثقة (Security Model & Trust Boundaries)

6.1 مناطق الثقة الخمس (Trust Zones)

- **المنطقة 1 — الشبكة الصفّية الخارجية (Classroom LAN)**: غير موثوقة؛ المنفذ الوحيد المعرّض منها نحو المعمل (عند تفعيل وضع `live-fire` فقط) هو 2222/TCP عبر قواعد تحويل الوجهة (`enable-live-fire.sh:136–143`).
- **المنطقة 2 — النظام المضيف (Physical Host)**: أعلى منطقة ثقة؛ يستمع خادم الموافقة `approval_daemon.py` حصرياً على عنوان الجسر الداخلي `192.168.122.1` ويرفض الارتباط بالعنوان العام `0.0.0.0` (`approval_daemon.py:567–579`).
- **المنطقة 3 — الشبكة المعزولة `virbr1` (10.10.0.0/24)**: معزولة تماماً بلا توجيه (`forwarding`) ولا DHCP (`backend-isolated.xml:6–18`)، وتعمل فيها آلة الفخّ على العنوان `10.10.0.10` دون بوابة افتراضية (`cloud-init-vm1/network-config:9–21`).
- **المنطقة 4 — شبكة `virbr0` الخاصة (192.168.122.0/24 NAT)**: تملك الآلة `vm-soc` واجهة `eth0` عليها (لأغراض سحب صور الحاويات والتحديثات والاتصال بالمضيف) إلى جانب واجهتها الثانية `eth1` على الجسر المعزول `virbr1` (`cloud-init-vm2/network-config:10–33`).
- **المنطقة 5 — عزل الحاويات (Container Hardening)**: تخضع حاوية الفخّ لضوابط `cap_drop ALL` و`no-new-privileges` و`pids_limit 256` (`vm1-honeypot/docker-compose.yml:44+`).

6.2 الضوابط الصارمة في طبقة التنفيذ

- **استدعاء `virsh` بلا وسيط `Shell`**: تُمرَّر الأوامر كمصفوفة معاملات `argv` مباشرة (`approval_daemon.py:137–151`)، وتنحصر قائمة الأوامر المسموح بها في عمليتين اثنتين فقط: أخذ لقطة `snapshot-create-as` وتعطيل واجهة الشبكة `domif-setlink down` (`:349–356`)، بينما تُعرَض الأوامر المقترحة الواردة في `proposed_commands` للمطالعة فحسب ولا تُنفَّذ أبداً (`:332–336`).
- **التحقق المتعدد الطبقات على المسار `/alert`**: يشمل التحقق من صحة عنوان IP عبر `ipaddress` (`:240–244`)، ومطابقة اسم الآلة الافتراضية مع النطاق المعتمد (`:246–247`)، وتقييد مدى `severity` (`:249–254`)، ومطابقة `alert_id` بتعبير نمطي (`:263–266`)، وتحديد سقف حجم الطلب بـ `64 KiB` (`:390`)، واستخدام طابور محدود بسعة 12 طلباً يرجع `HTTP 202` (`:452–455`).
- **إلزامية القرار البشري**: يعتمد `Confirm.ask` القيمة الافتراضية الرافضة `False` (`:478–482`)؛ ولا يعمل وضع الموافقة التلقائية `auto_approve` إلا بتفعيل صريح عبر البيئة (`:472–474`)، مع توفّر وضع المحاكاة `dry-run` (`:358–364`).
- **سجل التدقيق الإلزامي**: يُحفَظ كل قرار في `state/incidents.jsonl` مع فرض `fsync` (`:177–186`) — فلا يُنفَّذ أي إجراء دون أثر جنائي مسجّل.

6.3 المخاطر المتبقية والتوصيات الأمنية

⚠ **تنبيه أمني تشغيلي**: قبل تشغيل المعمل في بيئة صفّية مشتركة (`Live-Fire`)، يلزم تعيين قيمة قوية للمتغير `SENTINEL_TOKEN` وتغيير كلمة مرور Grafana الافتراضية (`admin/sentinel`).

- **رمز `SENTINEL_TOKEN` فارغ افتراضياً** (`vm2-soc/docker-compose.yml:149`): يعني ذلك غياب المصادقة التشفيرية على نقطة النهاية `/alert` في الإعداد الافتراضي، ويبقى الحاجز الحامي هو التحقق من المخطط والقرار البشري التفاعلي؛ لذا يُوصى بضبطه إلزامياً قبل أي استخدام صفّي.
- **بيانات الدخول الافتراضية لـ Grafana هي `admin/sentinel`** (`vm2-soc/docker-compose.yml:56–58`): يجب تغييرها قبل فتح المعمل للطلاب.
- **خدمة Loki تعمل بلا مصادقة** (`auth_enabled: false` في `loki/loki-config.yml:1–80`): وهو إعداد مقبول حصراً لأن المنفذ 3100 محصور داخل الجسر المعزول.

- **قياس العتبة وفترة التهدئة (120s) لكل عنوان IP على حدة**: الهجوم الموزّع البطيء عبر عناوين مصدرية متعددة قد يبقى دون عتبة الإطلاق (5 محاولات/30 ثانية لكل عنوان)، لأن النافذة تُحسَب لكل IP منفرداً لا على مستوى الفخّ إجمالاً (agent_detecteur.py:95–123).
- **مخاطر حقن المطالبة (Prompt Injection) عبر أسماء المستخدمين في السجلات**: أثرها محصور معمارياً، لأن وكيل التحليل يفرض عقد مخرجات صارماً ويحصر الإجراءات ضمن ALLOWED_ACTIONS (:35)، كما أن خادم الموافقة يبني أوامر virsh داخلياً ولا يشتقها من مخرجات النموذج اللغوي (approval_daemon.py:318–336).

7. تحليل الفشل والمرونة التشغيلية (Failure & Resilience Analysis)

7.1 سياسات إعادة المحاولة (Retry & Backoff Policies)

- **حلقة استقصاء الوكلاء**: تطبق تراجعاً أُسّياً يبدأ من ثانية واحدة ويتضاعف عند كل خطأ حتى سقف 30 ثانية (main.py:197–215).
- **شاحن السجلات promtail**: يطبق تراجعاً أُسّياً من 500ms إلى 5 دقائق بحدّ أقصى 20 محاولة (promtail/promtail-config.yml:24–27).
- **إرسال التقارير في rapporteur**: تنفّذ الدالة deliver أربع محاولات بتراجع أُسّي 2^attempt، وتمتنع عن التكرار عند تلقي رموز 4xx (agent_rapporteur.py:172–206).
- **خادم الموافقة controller**: يستوعب الطابور ذو السعة 12 طلباً الذروات المفاجئة بإرجاع HTTP 202 فوراً (approval_daemon.py:452–455)، بينما تمنع فترة التهدئة البالغة 120 ثانية لكل مصدر (:442–450) إغراق المشغّل بالتنبيهات المتكررة.

7.2 دلالات إعادة الإقلاع واستمرارية الحالة

- **موضع قراءة promtail في /tmp** (promtail/promtail-config.yml:17–18): قد تؤدي إعادة تشغيل الحاوية إلى إعادة شحن سجلات سبق إرسالها؛ ويعمل طابور بصمات SHA-1 ذو السعة 20,000 في agent_parseur.py:124, 191– 194 كخط دفاع لمنع تكرار معالجتها.
- **المؤشر الزمني last_ns**: يُحفَظ ذرّياً عبر os.replace (:146–155) ويُستعاد عند الإقلاع (:128–144)؛ أما البنى المحفوظة في الذاكرة العشوائية (طابور منع التكرار، ونوافذ الكشف المنزلقة، وخريطة التهدئة) فتُهيَّأ من جديد عند إعادة تشغيل حاوية الوكلاء.
- **توقيت حفظ المؤشر**: يُكتَب last_ns بعد انتهاء معالجة الدفعة (main.py:231–232) — وهي مقايضة تصميمية مشروحة في القسم 9.

7.3 أنماط التدهور الآمن (Graceful Degradation)

- **تعطّل Ollama أو بطء الاستجابة**: تنتهي مهلة الانتظار في analyste فينتقل التنفيذ فوراً إلى الدالة الاستدلالية الحتمية (agent_analyste.py:231–254) دون توقف خط الكشف؛ ويوثّق ملف README نجاح اختبار شامل (E2E) بتاريخ 2026-03-11 عبر هذا المسار الاحتياطي تحديداً.
- **تعطّل Loki**: يواصل promtail إعادة المحاولة حتى 20 مرة (promtail/promtail-config.yml:24–27)، في حين يدخل وكيل الفرز في التراجع الأُسّي للحلقة الرئيسية (main.py:197–215).
- **تعطّل خادم الموافقة controller**: تنهي الدالة deliver محاولاتها الأربع دون تجميد الحلقة الرئيسية، وتظل تقارير الحادثة بصيغتَي .md و.json محفوظةً بأمان على القرص (agent_rapporteur.py:209–238)؛ ولا يتم تنفيذ أي عزل تلقائي في غياب المشغّل — بحيث تبقى آلة الفخّ متاحةً كوضع افتراضي آمن.
- **تعطّل Grafana**: لا يتأثر خط الكشف أو الاستجابة مطلقاً، نظراً لأن دورها يقتصر على العرض البصري.

7.4 توثيق سلوك الفشل في الشفرة

- توثّق سلسلة التوثيق (docstring) في مطلع `main.py:11–18` ضمانات التشغيل الأساسية: النبض الدوري (heartbeat)، والاستقصاء (poll)، والتراجع الأُسّي (backoff)، والتسخين المسبق (warm-up).

7.5 إجراءات الاستعادة بعد الاحتواء

- لإعادة آلة الفخّ إلى العمل بعد عزلها: يُنفَّذ الأمر `../host/controller/restore-honeypot.sh --revert`، الذي يستعيد اللقطة `clean-baseline` مع إبقاء الآلة قيد التشغيل (`snapshot-revert --running` في `:126`) ويعيد رفع واجهة الشبكة (`domif-setlink up` في `:131`).
- للتحقق من نجاح الاستعادة وسجل القرارات: يُفحَص خرج `virsh snapshot-list vm-honeypot` وآخر السطور في ملف التدقيق `host/controller/state/incidents.jsonl`.

8. دليل التشغيل العملي (Operational Runbook)

8.1 بناء المعمل وتشغيله من الصفر

```
sudo nixos-rebuild switch --flake ./host/nixos#default
./host/provisioning/deploy-lab.sh
python3 host/controller/approval_daemon.py
ssh soc@192.168.122.20 'sudo docker compose -f /opt/soc/docker-compose.yml up -d'
cp .env.example .env   # يُنفّذ مرة واحدة؛ لضبط DASH_VM*_PASSWORD أو مفتاح SSH
./scripts/tmux-dashboard.sh
```

8.2 العمليات التشغيلية اليومية

- **إطلاق هجوم محاكاة يدوي**: عبر الأمر `ssh soc@10.10.0.10 'sudo docker compose -f /opt/honeypot/docker-compose.yml run --rm simulator'` باستخدام بيانات الحساب `soc:sentinel` (`cloud-init-vm1/user-data:34`)، حيث يستقر المشروع في المسار `/opt/honeypot` كما هو مُعرَّف في خطوات `runcmd` (`:120`).
- **تفعيل وضع الاشتباك الصفّي (Live-Fire)**: عبر الأمر `sudo ./host/provisioning/enable-live-fire.sh` متبوعاً بالأمر الفرعي `status` للتحقق؛ وعندئذٍ يتمكن الطلاب من مهاجمة المنفذ 2222 على عنوان المضيف مباشرةً (`:136–143`).
- **التعامل مع التنبيهات الأمنية**: تعرض نافذة `controller` تفاصيل الأدلة الجنائية وتنتظر إدخال `[y/N]`؛ وعند الموافقة بإدخال `y`، تُنشأ لقطة `snapshot` ذرّية وتُعطَّل واجهة الشبكة `NIC` مع تدوين القرار في سجل التدقيق عبر `append_audit` و`fsync`. وللتحقق من الأثر الجنائي يُستخدَم الأمر `virsh snapshot-list vm-honeypot` والأمر `tail` على الملف `host/controller/state/incidents.jsonl`.
- **استعادة الفخّ بعد العزل**: عبر تشغيل `../host/controller/restore-honeypot.sh --revert`.
- **إيقاف المعمل أو هدمه**: يُشغَّل `./host/provisioning/shutdown-lab.sh --dry-run` للمعاينة ثم بدون خيار المحاكاة لإيقاف الحزم والآلتين الافتراضيتين مع حفظ حالتهما؛ أما `./host/provisioning/cleanup-lab.sh` فهو أداة الهدم الكامل المُتعمَّد.
- **المراقبة الدورية والإيقاف الشامل للوحة**: عبر `./scripts/dashboard-status.sh --watch 5` للمتابعة الحية، و`./scripts/tmux-dashboard-stop.sh` مع الخيارين `--stop-stacks` و`--stop-vms` للإغلاق الكامل.

8.3 جدول المنافذ الشبكية

المنفذ	الخدمة والوجهة الشبكية	المصدر المرجعي
2222/TCP	تحويل وجهة DNAT على المضيف نحو 10.10.0.10:2222 (عند تفعيل live-fire فقط)	enable-live-fire.sh:136–143
3100/TCP	خدمة Loki على الواجهة eth1 للآلة 10.10.0.20 (vm-soc) داخل الجسر المعزول	loki/loki-config.yml:1–80
3000/TCP	واجهة Grafana على العنوان 192.168.122.20 (متاحة من النظام المضيف)	vm2-soc/docker-compose.yml:47–78
11434/TCP	محرك Ollama للاستدلال اللغوي داخل vm-soc	vm2-soc/docker-compose.yml:89–92
5000/TCP	خادم الموافقة approval_daemon على عنوان المضيف 192.168.122.1	approval_daemon.py:388–461
9999/TCP	منفذ إشارة الجاهزية الأولية (beacon) على 10.10.0.10 (VM1) أثناء التزويد	cloud-init-vm1/user-data:75–96

8.4 مسارات المخرجات والأدلة الجنائية (Artifacts)

- **تقارير الحوادث**: يُحفَظ الملفان incident-<STAMP>-<IP>.md وjson. لكل حادثة داخل مجلد reports على الآلة vm2-soc بملكية المعرّف 10002 (UID cloud-init-vm2/user-data:78).
- **سجل التدقيق الأمني (Audit Ledger)**: يُحفَظ في host/controller/state/incidents.jsonl مع فرض fsync لكل سجل (approval_daemon.py:105–186).
- **سجلات لوحة التحكم**: تُحفَظ في logs/tmux-dashboard/ وملفات معرّفات العمليات في run/<session>/ (scripts/tmux-dashboard.sh:572–592).
- **اللقطات الافتراضية (Snapshots)**: تشمل اللقطة المرجعية (deploy-lab.sh:554–565 clean-baseline) ولقطات الأدلة الجنائية (approval_daemon.py:349–355 forensic-breach-UTC-STAMP).

8.5 متغيرات البيئة الحاسمة

- SENTINEL_TOKEN: فارغ افتراضياً في ملف vm2-soc/docker-compose.yml:149 — ويلزم تعيينه وتمريره للطرفين قبل أي تشغيل رسمي (القسم 6.3).
- DASH_VM*_PASSWORD وDASH_SSH_KEY: متغيرات مصادقة SSH الخاصة بلوحة تحكم tmux.
- معاملات ضبط الكشف والاستدلال: POLL وLOOKBACK وTHRESHOLD وWINDOW وCOOLDOWN وOLLAMA_TIMEOUT وWEBHOOK في بيئة حاوية الوكلاء (vm2-soc/docker-compose.yml:134–150).

9. التفاوتات المرصودة والشفرة الخاملة وفرص التحسين

9.1 تفاوتات موثّقة بين الشفرة والوثائق

- **حجم ذاكرة vm-soc في ملخص التزويد**: تطبع بطاقة الملخص في نهاية deploy-lab.sh القيمة 5120 MiB للآلة vm-soc (:575))، بينما يبلغ التخصيص الفعلي في أمر virt-install للآلة الثانية (:459 4096 MiB) — فالرقم 5120 هو مجموع ذاكرة الآلتين معاً (4096 + 1024) وليس ذاكرة vm-soc وحدها.
- **وصف عتبة التنبيه في لوحة Grafana**: يذكر الوصف النصي في اللوحة العبارة «more than 5 failed logins» (ssh-soc-dashboard.json:26, 313)، في حين أن الشرط الفعلي في الشفرة البرمجية هو 5 =>

((agent_detecteur.py:130).

- **مراجع قديمة في `TROUBLESHOOTING.md`**: لا يزال الملف يحتفظ بأسطر تكوين قديمة لـ `promtail` وبعنوان ثابت `172.31.240.66`، بينما تُعرَّف شبكة `vm1` كشبكة فرعية `172.31.240.0/24` دون عناوين ثابتة للحاويات (`vm1-honeypot/docker-compose.yml:116`).
- **القيمة الافتراضية لـ `OLLAMA_TIMEOUT`**: يذكر ملف `CHANGELOG.md:43` أن المهلة خُفِّضت من 180 إلى 25 ثانية — غير أن القيمة الافتراضية في الشفرة المصدرية ما زالت 180 ثانية (`main.py:109`)، ولا تُخفَّض إلى 25 إلا عبر متغير البيئة الممرَّر في `vm2-soc/docker-compose.yml:134–150`.
- **مكوّن `ruler` الخامل في Loki**: يظهر قسم `ruler` مفعّلاً في `vm2-soc/loki/loki-config.yml:1–80` دون أي ملفات قواعد مرفقة — وهو تكوين خامل لا يؤدّي وظيفة حالياً.
- **وصول الإنترنت في `ARCHITECTURE.md`**: تذكر الوثيقة عبارة «Internet access for both VMs»، وهو ما يصحّ لآلة الفخّ `VM1` فقط خلال نافذة التزويد الأولي قبل أن يقوم السكربت بفصل بطاقة الشبكة المؤقتة (`deploy-lab.sh:535–552`).

9.2 مواطن هشاشة هندسية قابلة للتقوية

- **تخزين `positions.yaml` في `/tmp`** (`promtail/promtail-config.yml:17–18`): يؤدي إقلاع الحاوية إلى إعادة قراءة السجلات، ويمثّل طابور `SHA-1` في الوكيل خط تخفيف ولكنه محدود بـ `20,000` حدث.
- **خريطة التهدئة (`cooldown map`) في خادم الموافقة**: لا تخضع لعملية تقليم دوري للعناوين المنتهية (`approval_daemon.py:442–450`)، مما يسبب نمواً طفيفاً في الذاكرة عند التشغيل الطويل جداً.
- **نافذة تثبيت المؤشر الزمني** (`main.py:231–232`): بما أن المؤشر `last_ns` يُحفَظ بعد معالجة الدفعة كاملة، فإن حدوث استثناء غير متوقع أثناء معالجة دفعة متعددة التنبيهات قد يسبّب إعادة معالجة بعض أحداث الدفعة أو تأخرها.
- **فحوص الصحة في لوحة التحكم**: تتحقق الفحوص في `scripts/dashboard-status.sh:180–454` من وجود العمليات واستجابة المنافذ، لكنها لا تقيس حداثة طابع النبض `heartbeat` — فتكتشف توقف الخدمة الكامل لكنها قد لا ترصد تجمّد الحلقة الداخلية ببطء.
- **وضع صيانة `Promtail`**: يشير قسم القضايا المعروفة في `README` إلى انتقال `Promtail` لوضع الصيانة، مع التوصية بالترقية مستقبلاً إلى `Grafana Alloy` متى ما سمحت ميزانية الذاكرة.

9.3 قائمة التحسينات المقترحة

- إلزامية ضبط `SENTINEL_TOKEN` بطول تشفيري كافٍ ورفض الإقلاع في الوضع الصفّي إذا تُرك فارغاً (القسم 6.3).
- تأمين اتصال `Webhook` عبر `HTTPS/mTLS` بدلاً من `HTTP` الصريح عبر جسر `virbr0`.
- إضافة نافذة كشف تجميعية على مستوى الهدف (`Target-wide Sliding Window`) لرصد هجمات التخمين الموزّعة (`Distributed Brute Force`) عبر عناوين `IP` متعددة (القسم 6.3).
- كتابة اختبارات وحدة (`Unit Tests`) لمعادلة حساب الخطورة في `_build_alert` ولعقد المخرجات في `agent_analyste.py`.
- تقليم خريطة `cooldown` دورياً في خادم الموافقة، وربط فحوص الصحة في اللوحة بعمر آخر نبض `heartbeat` فعلي.

10. مسرد المصطلحات والمجالات غير المتحقق منها

10.1 مسرد المصطلحات التقنية والمعمارية

- **`Parseur` (وكيل الفرز)**: المرحلة الأولى من خط الأنابيب؛ تحوّل أسطر `Loki` الخام إلى أحداث `Event` مصنّفة وخالية من الضجيج (`agent_parseur.py:40–46`).
- **`Détecteur` (وكيل الكشف)**: المرحلة الثانية؛ تطبق نافذة زمنية منزلقة وعتبة إخفاق لإطلاق تنبيه `Alert` ((`agent_detecteur.py:112–141`).

- **`Analyste` (وكيل التحليل)**: المرحلة الثالثة؛ تثري التنبيه عبر النموذج اللغوي في `Ollama` مع آلية استدلال احتياطية حتمية (`agent_analyste.py:152–195، :231–254`).
- **`Rapporteur` (وكيل التوثيق والإشعار)**: المرحلة الرابعة؛ تكتب تقريرَي `.md` و`.json`. وترسل إشعار `Webhook` إلى خادم الموافقة (`agent_rapporteur.py:70–238`).
- **`live-fire` (وضع الاشتباك الحي)**: وضع التدريب الصفّي الذي يمرّر هجمات الطلاب عبر قواعد `DNAT` على واجهة المضيف (`enable-live-fire.sh:130–143`).
- **`virbr1` (backend-isolated)**: الجسر الشبكي المعزول `10.10.0.0/24` الخالي من التوجيه ومن `DHCP` (`backend-isolated.xml:22–27`).
- **`forensic-breach-UTC-STAMP`**: اسم لقطة الأدلة الجنائية التي تُنشأ ذرّياً فور موافقة المشغّل وقبل فصل الشبكة (`approval_daemon.py:349–355`).
- **`clean-baseline`**: اللقطة المرجعية النظيفة التي تُؤخذ عقب انتهاء التزويد الأولي ويُعاد الفخّ إليها عند الاستعادة (`deploy-lab.sh:554–565`).
- **`beacon` (منارة الجاهزية)**: خدمة استماع مؤقتة على المنفذ `9999/TCP` تعلن اكتمال تهيئة `VM1` أثناء التزويد (`cloud-init-vm1/user-data:75–96`).
- **`cidata`**: معرّف صورة القرص `ISO` التي تحمل بيانات تهيئة `NoCloud` (`cloud-init`) لكل آلة افتراضية (`deploy-lab.sh:392–417`).
- **`domif-setlink`**: أمر `virsh` المستخدم لفصل واجهة الشبكة الافتراضية (`down`) أو إعادة وصلها (`up`) (`approval_daemon.py:356`).

10.2 المجالات غير المتحقق منها سكونياً

- الملفات الثنائية (`*.qcow2`، وصور `ISO`، وملفات `*.pyc`) لم تُفحَص محتوياتها مباشرةً؛ إذ يعتمد التحقق منها على بيئة التشغيل الفعلي.
- لم يُشغَّل المعمل حياً خلال جلسة المراجعة الساكنة هذه؛ وتُعدّ ملاحظة التحقق الشامل (`E2E`) المؤرخة في `2026-11-03` داخل `README` قرينةً توثيقية مساندة لا بديلاً عن الفحص الحي المستقل.
- مسار تشغيل خادم الموافقة داخل حاوية `Docker` المشار إليه في القسم `§8.1` من الوثيقة `docs/01-INSTALL-NIXOS.md` لم يُتحقَّق منه سطراً بسطر.
- المكونات الداخلية لصورة حاوية `ollama` (إصدارات المكتبات الديناميكية وسلوك وقت التشغيل الدقيق) تقع خارج نطاق الشفرة المصدرية للمستودع.
- تم تحويل مخططَي `Mermaid` الواردين في القسمين `2.2` و`2.3` وتضمينهما كرسوم متجهة عالية الدقة (`architecture-diagram.svg` و`sequence-diagram.svg`) لضمان اكتمال العرض البصري المباشر داخل الوثيقة المترجمة.

---

i منهجية التحقق المستقل: كل إشارة مرجعية بالصيغة `path:line` في هذا التقرير قابلة للفحص المباشر عبر أوامر البحث القياسية في المستودع المصدري، مثل: `grep -n "snapshot-create-as" host/controller/approval_daemon.py` أو `grep -n "forensic-breach" host/controller/approval_daemon.py`. وأي ادعاء لا يطابق السطر المصدري المشار إليه يُعدّ خطأً في التقرير لا في النظام.
